

App Summary: Underwriting Tool Backend

Repo-based one-page overview

What It Is

A FastAPI backend for onboarding users and processing bank statement analysis workflows. It accepts PDF statement uploads, sends them to ScoreMe for analysis, stores workflow state in MongoDB, links accounts in PostgreSQL, and ingests returned reports.

Who It Is For

Primary persona: a business customer or loan applicant using an underwriting portal to register, log in, and upload bank statements for review. Internal underwriting or ops teams likely consume the resulting data, but that user flow is not found in repo.

What It Does

- Registers users, creates linked account records, and emails login credentials.
- Authenticates users with JWT stored in an HTTP-only cookie.
- Returns the current authenticated user profile.
- Accepts up to 12 PDF bank statements plus account metadata per upload request.
- Submits uploads to the ScoreMe bank statement analysis API.
- Stores upload metadata and original PDFs in Google Cloud Storage and MongoDB.
- Consumes webhook/report URLs, updates reference records, and saves normalized bank report data.

How It Works

- Client -> FastAPI routes ('/v1/auth', '/v1/user', '/v1/bsa') with auth middleware on non-public paths.
- FastAPI lifespan opens MongoDB and PostgreSQL connections and stores them on 'app.state'.
- Auth flow writes user/auth documents to MongoDB and may create/find an 'accounts' row in PostgreSQL.
- BSA upload flow validates PDFs and metadata, sends files to ScoreMe, and stores a 'bsa_reference' record in MongoDB.
- Background task uploads source PDFs to Google Cloud Storage and stores file metadata in 'bsa_file_uploads'.
- Webhook endpoint updates the reference record, then fetches the ScoreMe JSON report and stores an optimized document in 'bankstatementreport'.
- Celery app exists with Redis config and 'tasks.bsa_tasks' import, but active worker startup usage is not found in repo.

How To Run

- Create and activate a Python virtual environment.
- Install dependencies: 'pip install -r requirements.txt'.
- Set required environment variables seen in code: 'MONGO_URI', 'POSTGRES_URI', 'JWT_SECRET_KEY', 'JWT_ALGORITHM', 'CLIENT_ID', 'CLIENT_SECRET', 'GCS_BUCKET_NAME', 'APP_PASSWORD', 'ACCOUNT_OWNER_ID', and 'CREATED_BY_ID'.
- Provide Google Cloud credentials for the storage client. Exact setup method: Not found in repo.
- Start the API with 'python main.py' and it serves on port 8080.
- Optional infra implied by code: MongoDB, PostgreSQL, ScoreMe access, SMTP, and likely Redis/Celery; exact local bootstrap commands are Not found in repo.